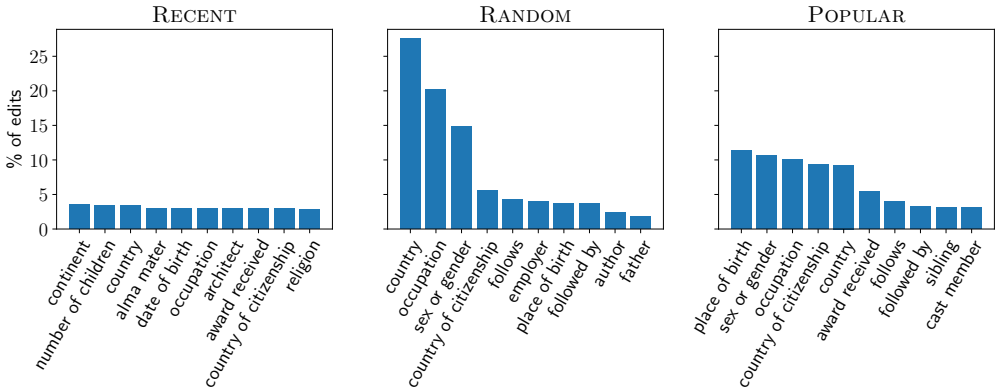

Figure 4: Most frequent relations and their frequency, in each subset of RIPPLEEDITS.

	RECENT	RANDOM	POPULAR
# of factual edits	2,000	1,000	1,000
# of queries per edit	26.8	18.8	25.6
# of queries per criterion	5.24	3.1	4.2
# of LG queries	2.5	3.6	2.6
# of CI queries	11.7	4.7	6.1
# of CII queries	5.1	5.1	3.9
# of SA queries	1.8	1.3	4.7
# of PV queries	0.6	0.4	0.5
# of RS queries	5.1	3.7	7.8
Subject triplets count	31.7	13.3	115.2
Subject page back-links	278.1	121.6	3934.5
Subject page views	189.6	67.91	7376.5
Object triplets count	192.4	46.4	39.5
Object page back-links	18634.2	3065.0	2136.0
Object page views	2852.4	1379.7	1176.7

Table 1: Statistics per subset of RIPPLEEDITS, showing the average of different metrics. Breakdown by evaluation criteria shows the number of queries of each criterion per edit. For a given subject/object entity, triplets count is the number of WIKIDATA facts it is associated with, page back-links is the number of Wikipedia pages with a link to the entity's page, and page views is the recent average daily view count of the entity's page.

inputs to LMs, we convert them from triplet-form to natural language (NL). To this end, we manually craft a template NL phrase per relation (this is feasible since we use a fixed set of relations), and use it to convert all the triplets with this relation. For instance, the template "`The date of birth of <e> is`" converts triplets with the relation $r =$ `Date of Birth` and a subject entity e.

For the *Preservation* triplets generated for an edit $(e, r, \{o_1, ..., o_n\}) \rightarrow (e, r, \{o_1, ..., o_n, o^*\})$, where o^* is a new object added to a set of possibly multiple ($n \geq 0$) objects, we form a single NL query about other objects than the edited one, e.g., "`The award received by <e> which is not <o*> is`".

4.2 Data Statistics

We used our data generation pipeline to collect edits for 2,000 RECENT facts, 1,000 RANDOM facts, and 1,000 POPULAR facts, focusing on $N_{rel} = 54$ basic relations for which we manually crafted NL templates and logical rules.[4] To obtain the RANDOM subset, we set $N_{ent} = 200$ to sample 200 facts from each entity group in WIKIDATA. For edits generation of RANDOM and POPULAR, we set $N_{cand} = 100,000$. We call our diagnostic benchmark RIPPLEEDITS, and publicly release it to the research community. Notably, RIPPLEEDITS focuses on ripple edits and is meant to complement existing benchmarks, and so it does not include previous evaluations, such as subject specificity and model consistency.

Statistics on RIPPLEEDITS are presented in Table 1, showing that our generation process resulted in 18-26 test queries per edit and over 3 queries per evaluation test, on average. Moreover, POPULAR edits contain more popular subjects (as intended), while RECENT edits have more popular objects. Fig. 4 shows the top relations and their frequency in each subset of RIPPLEEDITS, demonstrating the diversity of the generated facts.

4.3 Data Quality

We conducted a manual analysis to validate that our generation pipeline produces valid test queries. Concretely, we sampled 200 random test queries from RIPPLEEDITS and checked the following two requirements: (a) *soundness*: the triplet that represents a given test query should be semantically correct, namely, the entity type of the object should match the relation type and the relation type should

[4]We release the templates and rules in our codebase.

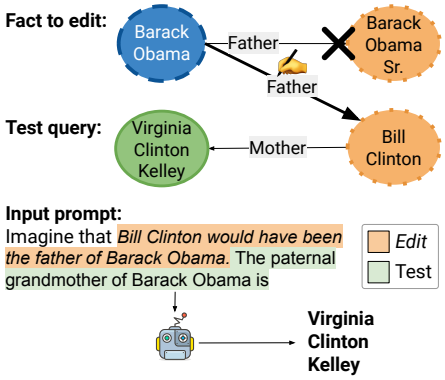

Figure 5: An example modification edit from our ICE baseline. The color code of the KG is similar to that described in Fig. 2. We prepend the prefix "*Imagine that*" to the input prompt, as counterfactuals can contradict knowledge embedded in a model's parameters.

match the entity type of the subject. For example, queries such as *"The capital of Hilary Clinton is"* or *"The sibling of Lebron James is Los Angeles"* would have been disqualified. (b) *grammatically correct*: we check that the phrasing of the test query in natural language is grammatical.

We found that 100% of the queries were sound (i.e., semantically clear and correct), showing that the data curating process was designed properly. Furthermore, 98.5% of the queries were grammatically correct, while the ones which were not contain entity representations in a non-English language. This shows that our templates are general enough to properly fit various entity names.

5 Experiments

We use RippleEdits to evaluate recent KE methods, and show that despite substantial progress on existing benchmarks, current methods struggle to introduce consistent changes to the model's knowledge after an edit. Moreover, a simple in-context editing baseline that conditions the generation on the edited fact obtains better results, while leaving ample room for improvement for future research.

5.1 Evaluation Setting

Data To evaluate how well an editing method handles the ripple effects resulting from editing a given model, the data first needs to be adjusted such that (a) only cases of successful edits are evaluated, and (b) only test queries that the model answered correctly pre-editing are used for evaluation. Concretely, for an editing method $\mathcal{F}$ and a model $\mathcal{M}$, an edit request $x : (e, r, o) \rightarrow (e, r, o')$ is included in the evaluation if the following conditions are met when applying $\mathcal{F}$ to $\mathcal{M}$ and x: (a) $\mathcal{M}$ successfully generates the original objects for the test queries before applying the edit, and (b) $\mathcal{M}$ successfully generates o' when queried about e and r, namely, the edit has successfully been applied. For example, we verify that the model can predict the children of o' before asking about e's new siblings.

	Recent		Random		Popular	
	Edits	Tests	Edits	Tests	Edits	Tests
GPT-2	853	29%	689	33%	722	71%
GPT-J	801	33%	717	34%	760	76%
GPT-Neo	989	45%	801	46%	828	86%
LLaMA	847	44%	796	49%	784	87%
GPT-3	822	55%	760	74%	665	94%

Table 2: (a) Number of edits considered in our evaluation (i.e., that have successfully applied), from each subset, averaged over ROME, MEMIT and MEND, for the models: GPT-2, GPT-J, GPT-Neo and LLaMA, and the ICE baseline for GPT-3. (b) Portion of queries, on average, that were used in our evaluation.

Editing methods We evaluate three KE methods: MEND (Mitchell et al., 2022b), ROME (Meng et al., 2022), and MEMIT (Meng et al., 2023). MEND trains a network that modifies gradients to produce local edits. ROME makes rank-one updates to the weights of the Transformer's MLP layers to modify specific factual associations, and MEMIT is an extension of ROME that is adjusted to editing many facts at once.

Baseline Motivated by the recent success of LMs to learn in-context and follow instructions (Brown et al., 2020a; Ouyang et al., 2022; Liu et al., 2023), specifically for knowledge editing (Zhong et al., 2023; Zheng et al., 2023), we experiment with an in-context editing (ICE) baseline for factual editing. Unlike the above methods, it does not introduce changes to the model parameters, but rather generation is conditioned on the new fact. Concretely, given an edit $(e, r, o) \rightarrow (e, r, o^*)$ and a test query q, we use the following prompt to obtain an answer from the model: "`Imagine that <`o^*`> would have been <`P_r`>`", where P_r is a manually-written proposition of r, such as *"The mother of* `<`e`>`*"* when r = `Mother` and e is the subject. An example is depicted in Fig. 5.

Models We use 4 recent auto-regressive decoder-only LMs of different sizes: GPT-2 XL (Radford et al., 2019) with 1.5B parameters, GPT-J (Chen et al., 2021) with 6B parameters, LLaMA with 7B parameters, (Touvron et al., 2023), and GPT-NeoX

		LG	CI	CII	SA	PV	RS	Avg.
GPT-2	ROME	20.2	35.6	46.8	86.8	100	55.4	57.5
	MEMIT	21.8	30.3	46.2	92.9	100	56.8	58.0
	MEND	28.9	23.7	20.7	87.1	100	51.9	52.1
GPT-J	ROME	15.2	29.5	50.5	90.3	99.4	60.0	57.5
	MEMIT	18.0	35.0	48.1	88.4	98.6	42.2	55.0
GPT-NEO	ROME	27.2	54.3	69.4	98.9	98.4	80.3	71.4
	ICE	48.3	29.0	62.2	100	99.4	80.7	69.9
LLAMA	ROME	16.7	47.8	50.0	93.6	97.6	59.3	60.8
	ICE	59.6	74.8	85.0	100	99.5	77.9	82.8
GPT-3	ICE	33.3	100	91.3	100	100	73.1	82.8

Table 3: Accuracy on the RECENT subset, by MEND, ROME, MEMIT, and the ICE baseline, on GPT-2, GPT-J, GPT-NEO, LLAMA, and GPT-3.

		LG	CI	CII	SA	PV	RS	Avg.
GPT-2	ROME	53.6	31.6	44.4	94.9	9.9	38.9	45.5
	MEMIT	58.4	30.5	49.8	100	20.0	36.2	49.1
	MEND	62.5	16.7	14.6	91.3	17.7	30.1	38.8
GPT-J	ROME	53.8	40.8	49.9	93.8	15.2	39.4	48.8
	MEMIT	53.0	35.7	48.2	95.6	18.2	39.9	48.4
GPT-NEO	ROME	61.6	49.4	57.1	100	30.8	50.7	58.3
	ICE	78.6	90.0	55.6	100	100	61.9	81.0
LLAMA	ROME	54.3	35.5	49.5	96.0	17.8	38.9	48.7
	ICE	71.1	73.8	80.3	100	100	69.6	82.5
GPT-3	ICE	69.0	83.3	89.7	100	100	100	90.3

Table 4: Accuracy on the RANDOM subset, by MEND, ROME, MEMIT, and the ICE baseline, on GPT-2, GPT-J, GPT-NEO, LLAMA, and GPT-3.

		LG	CI	CII	SA	PV	RS	Avg.
GPT-2	ROME	5.7	46.4	21.8	100	100	18.5	48.7
	MEMIT	6.7	45.2	21.2	100	100	24.3	49.6
	MEND	25.9	10.7	5.4	100	100	21.2	43.9
GPT-J	ROME	5.5	44.1	21.0	98.6	99.0	22.3	48.4
	MEMIT	7.0	45.9	23.7	100	100	24.8	50.2
GPT-NEO	ROME	36.4	29.4	41.6	100	100	50.8	59.7
	ICE	37.5	92.4	40.1	100	100	74.4	74.1
LLAMA	ROME	22.0	37.4	16.2	100	100	20.6	49.4
	ICE	57.2	85.1	67.6	100	100	78.0	81.3
GPT-3	ICE	31.0	86.1	65.6	100	100	83.8	77.7

Table 5: Accuracy on the POPULAR subset, by MEND, ROME, MEMIT, and the ICE baseline, on GPT-2, GPT-J, GPT-NEO, LLAMA, and GPT-3.

with 20B parameters (Black et al., 2022). In addition, as our baseline does not require access to the model parameters, we also evaluate it on the closed-source model GPT-3 `text-davinci-003` with 175B parameters (Brown et al., 2020b). However, for the baseline we do not include results for GPT-2 and GPT-J as the number of testable edits for these models is rather small ($\leq 20\%$ for each of the data subsets).

For all model-method combinations, except for ROME with LLAMA, we use the official implementation and hyperparameters from Meng et al. (2022). We adjust ROME to LLAMA by following the authors' method and codebase. Table 2 shows the number of edits and test queries left, for every model, after filtering out non-successful edits and inapplicable test queries (as described above).

Evaluation Each model-method pair is evaluated separately, on every subset of RIPPLEEDITS. For each evaluation criterion, we first compute the average accuracy over the test queries per example, and then average over all the examples. For a given test query, we let the model generate a maximum of 20 tokens. A generation is considered successful if one of the aliases of the target object appears in the text. In cases of multiple gold target objects (as in *Preservation*), we evaluate each target object separately and consider the generation as correct if it matches at least one object.

5.2 Results

Tables 3, 4, 5 show the evaluation results on the RECENT, RANDOM, and POPULAR subsets, respectively. Considering the average scores across all subsets, we observe that existing editing methods struggle to handle the ripple effect induced by editing facts, with low average accuracy of $38-66$ across all models. This suggests that, while KE methods demonstrate high capability in making local updates to the model's knowledge, these changes are mostly applied at a surface-level without propagating to other related facts. Moreover, we observe that our ICE baseline obtains the best overall results. Specifically, it outperforms ROME by more than 10 points for GPT-NEO and 29 points for LLAMA, on average across subsets. Although GPT-3 with ICE performs best on average, the 7B LLAMA is highly competitive, performing better or similarly on the RECENT and POPULAR subsets.

Next, comparing results across evaluation criteria shows that some ripple effects are handled better than others. For example, while *Subject Aliasing* accuracy is consistently high (≥ 86.8 across all settings), the accuracy on other criteria is generally lower and varies greatly between models, methods, and edits (e.g., *Logical Generalization* accuracy for ROME on GPT-J is 53.8 on the RANDOM subset, compared to only 5.5 on the POPULAR subset).